HTML

From Basic to Advanced

A Complete Student's Reference Guide

Chapters 1 – 41 · Concise Definitions · Working Code Examples

CHAPTER 1

Getting Started with HTML

Definition: HTML (HyperText Markup Language) is the standard markup language for creating web pages. It uses tags to structure content — not to style it. Styling is handled by CSS.

Core Concepts

• Element = opening tag + content + closing tag: Hello

• Void elements have no closing tag: <img>,
, <input>

• HTML5 is the current standard — always use <!DOCTYPE html>

Try It:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my first HTML page.</p>
</body>
</html>
```

Tip: Save as .html and open in any browser. The <head> holds metadata; <body> holds visible content.

CHAPTER 2

Anchors and Hyperlinks

Definition: The <a> (anchor) element creates clickable hyperlinks. The href attribute specifies the destination URL, page anchor, or protocol like mailto: or tel:.

Key Attributes

href – destination URL | target – where to open | rel – relationship | download – force download

Try It:

```
<!-- External link -->
<a href="https://example.com">Visit Example</a>

<!-- Open in new tab (add rel for security) -->
<a href="https://example.com" target="_blank" rel="noopener noreferrer">Open New
Tab</a>

<!-- Link to page anchor -->
<h2 id="section1">Section 1</h2>
<a href="#section1">Jump to Section 1</a>

<!-- Email link -->
<a href="mailto:hello@example.com?subject=Hi">Send Email</a>

<!-- Phone link -->
<a href="tel:+11234567890">Call Us</a>

<!-- Download link -->
<a href="file.pdf" download="myfile.pdf">Download PDF</a>
```

Tip: Always add rel="noopener noreferrer" when using target="_blank" to prevent security vulnerabilities.

CHAPTER 3

ARIA (Accessible Rich Internet Applications)

Definition: ARIA adds semantic meaning to HTML elements so assistive technologies (like screen readers) can describe interactive components to users with disabilities. Use native HTML elements first; add ARIA only when needed.

Common ARIA Roles & Attributes

▸ **Try It:**

```
<!-- Alert: announces urgent messages to screen readers -->
<div role="alert" aria-live="assertive">
  Your session expires in 60 seconds.
</div>

<!-- Dialog / Modal -->
<div role="dialog" aria-labelledby="dlgTitle">
  <h2 id="dlgTitle">Confirm Delete</h2>
  <button>Yes</button> <button>No</button>
</div>

<!-- Checkbox with ARIA state -->
<input type="checkbox" role="checkbox" aria-checked="false" id="agree">
<label for="agree">I agree to the terms</label>

<!-- Navigation landmark -->
<nav role="navigation" aria-label="Main menu">
  <ul>
    <li><a href="/">Home</a></li>
    <li><a href="/about">About</a></li>
  </ul>
</nav>

<!-- Progress bar -->
<progress role="progressbar" value="40" max="100">40%</progress>

<!-- Search region -->
<div role="search">
  <input role="searchbox" type="text" placeholder="Search...">
  <button>Go</button>
</div>
```

Tip: Do NOT add role="button" to a <button> — native elements already have implicit ARIA roles.

CHAPTER 4

Canvas

Definition: The <canvas> element is an HTML5 drawing surface. It is a container only — all drawing is done with JavaScript via the 2D rendering context (getContext("2d")).

Try It:

```
<!DOCTYPE html>
<html>
<body>
  <canvas id="myCanvas" width="400" height="200"
          style="border:1px solid #ccc;">
    Canvas not supported.
  </canvas>
  <script>
    const canvas = document.getElementById("myCanvas");
    const ctx = canvas.getContext("2d");

    // Red filled rectangle
    ctx.fillStyle = "red";
    ctx.fillRect(10, 10, 150, 100);

    // Blue stroked rectangle
    ctx.strokeStyle = "blue";
    ctx.lineWidth = 3;
    ctx.strokeRect(180, 10, 150, 100);

    // Text
    ctx.fillStyle = "black";
    ctx.font = "20px Arial";
    ctx.fillText("Hello Canvas!", 120, 170);
  </script>
</body>
</html>
```

Tip: Canvas is pixel-based and resolution-dependent. For scalable graphics, prefer SVG (Chapter 36).

CHAPTER 5

Character Entities

Definition: Character entities are special codes used to display reserved HTML characters (like < and >) or symbols not on a standard keyboard. They start with & and end with ;.

Common Entities

Try It:

```
<!-- Reserved characters -->
<p>5 &lt; 10 and 10 &gt; 5</p>
<p>Fish &amp; Chips</p>
<p>&quot;Quoted text&quot;</p>

<!-- Non-breaking space (keeps words together) -->
<p>10 kg</p>

<!-- Symbols -->
<p>&copy; 2024 My Company</p>
<p>&reg; Registered Trademark</p>
<p>&trade; Trademark</p>
<p>&mdash; em dash &ndash; en dash</p>

<!-- Numeric reference -->
<p>&#9742; Call us!</p>   <!-- Phone symbol -->
<p>&#128269; Search</p>  <!-- Magnifier -->
```

Tip: Always encode < as < and > as > inside HTML content to avoid browser parsing errors.

CHAPTER 6

Classes and IDs

Definition: class selects one or more elements for CSS styling (reusable). id uniquely identifies a single element on the page (must be unique — used for JavaScript and anchor links).

Try It:

```
<!-- HTML -->
<div class="card highlight">Styled Card</div>
<div class="card">Another Card</div>
<p id="main-intro">Unique paragraph</p>

<!-- CSS -->
<style>
  .card    { background: #e3f2fd; padding: 16px; margin: 8px; }
  .highlight { border-left: 4px solid blue; }
  #main-intro { font-size: 1.2em; color: darkblue; }

  /* Combining selectors */
  .card.highlight { font-weight: bold; }
</style>

<!-- Anchor link using ID -->
<a href="#main-intro">Jump to intro</a>
```

Tip: An element can have multiple classes (space-separated), but only ONE id. IDs must be unique per page.

CHAPTER 7

Comments

Definition: HTML comments are notes in your code that the browser ignores entirely. They are visible in the page source but do not render on screen.

Try It:

```
<!-- Single line comment -->

<!--
  Multi-line comment:
  This entire section is under construction.
  Remove this block when done.
-->

<h1>Hello <!-- this text is hidden --> World</h1>

<!-- Commenting out code temporarily: -->
<!-- <p>This paragraph is disabled for now</p> -->

<!-- IE Conditional Comments (IE 5-9 only) -->
<!--[if IE]>
  <p>You are using Internet Explorer.</p>
<![endif]-->
```

Tip: Comments cannot be nested. Do not put -- inside a comment body. End with --> not --->

CHAPTER 8

Content Languages

Definition: The lang attribute declares the human language of an element's content using BCP 47 language codes (e.g. "en" for English, "fr" for French). It helps screen readers pronounce content correctly and improves SEO.

Try It:

```
<!-- Declare the primary document language on <html> -->
<html lang="en">

<!-- Override language for a specific element -->
<p lang="fr">Bonjour le monde!</p>

<!-- Mixed language content -->
<p lang="en">
  The German word for world is <span lang="de">Welt</span>.
</p>

<!-- Attribute values in a different language -->
<p lang="en" title="A French greeting">
  <span lang="fr" title="Bonjour">Hello</span>
</p>

<!-- hreflang on links -->
<a href="https://example.fr" hreflang="fr">French version</a>
```

Tip: Always set lang on the <html> element. This is a WCAG 2.0 accessibility requirement (Success Criterion 3.1.1).

CHAPTER 9

Data Attributes

Definition: Data attributes (data-*) let you store custom information directly on HTML elements without affecting rendering. JavaScript accesses them via the dataset property.

Try It:

```
<!-- HTML: store custom data on elements -->
<div id="user"
     data-user-id="42"
     data-role="admin"
     data-active="true">
  John Doe
</div>

<button data-action="delete" data-target="user-42">Delete</button>

<!-- JavaScript: read and modify data attributes -->
<script>
  const el = document.getElementById("user");

  // Read
  console.log(el.dataset.userId);   // "42"
  console.log(el.dataset.role);     // "admin"

  // Write
  el.dataset.active = "false";

  // CSS can also target data attributes
  // [data-active="true"] { color: green; }
</script>
```

Tip: data-* names are hyphenated in HTML but camelCase in JavaScript dataset. data-user-id → dataset.userId

CHAPTER 10

Div Element

Definition: The element is a generic block-level container with no inherent semantic meaning. Use it to group elements for styling or layout when no semantic element (article, section, nav, etc.) fits.

Try It:

```
<!-- Basic grouping for styling -->
<div class="card">
  <h2>Card Title</h2>
  <p>Card content goes here.</p>
</div>

<!-- Nested divs (layout) -->
<div class="container">
  <div class="sidebar">Sidebar</div>
  <div class="main-content">Main</div>
</div>

<style>
  .container { display: flex; gap: 16px; }
  .sidebar   { width: 200px; background: #e3f2fd; padding: 8px; }
  .main-content { flex: 1; background: #f3e5f5; padding: 8px; }
  .card { border: 1px solid #ccc; padding: 16px; border-radius: 4px; }
</style>
```

Tip: Prefer semantic elements (<article>, <section>, <nav>, <main>) over when the content has meaning. Use as a last resort.

CHAPTER 11

Doctypes

Definition: The DOCTYPE declaration tells the browser which version of HTML the document uses. It must be the very first line of an HTML file (before the <html> tag). Without it, browsers enter "quirks mode" and may render incorrectly.

DOCTYPE Versions

Try It:

```
<!-- HTML5 (recommended — use this always) -->
<!DOCTYPE html>

<!-- HTML 4.01 Strict (no deprecated elements, no frames) -->
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
  "http://www.w3.org/TR/html4/strict.dtd">

<!-- HTML 4.01 Transitional (allows deprecated elements) -->
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
  "http://www.w3.org/TR/html4/loose.dtd">

<!-- HTML 4.01 Frameset (allows frames) -->
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN"
  "http://www.w3.org/TR/html4/frameset.dtd">

<!-- DOCTYPE is case-insensitive: all equivalent -->
<!DOCTYPE html>
<!doctype html>
<!DocType Html>
```

Tip: Always use <!DOCTYPE html> for new projects. HTML5 DOCTYPE is the simplest and works with all modern browsers.

CHAPTER 12

Embed

Definition: The <embed> element (HTML5) embeds external content such as plugins, PDFs, or multimedia directly into the page. The type attribute must specify the MIME type of the content.

Try It:

```
<!-- Embed a video file -->
<embed type="video/mp4" src="video.mp4" width="640" height="480">

<!-- Embed a PDF document -->
<embed type="application/pdf" src="document.pdf" width="800" height="600">

<!-- Embed an SVG image -->
<embed type="image/svg+xml" src="graphic.svg" width="300" height="300">

<!-- Embed an audio file -->
<embed type="audio/mpeg" src="song.mp3">
```

Tip: Prefer <video>, <audio>, or <object> for media. Use <embed> only when integrating legacy plugin-based content.

CHAPTER 13

Forms

Definition: The <form> element groups input controls and manages submission of user data to a server. The action attribute sets the destination URL; the method attribute sets GET or POST.

Try It:

```
<form action="/submit" method="post" autocomplete="on">

  <!-- Grouped fields with fieldset -->
  <fieldset>
    <legend>Personal Info</legend>
    <label for="fname">First Name:</label>
    <input type="text" id="fname" name="firstname" required><br>
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>
  </fieldset>

  <!-- File upload (must use multipart) -->
  <fieldset>
    <legend>Upload</legend>
    <input type="file" name="avatar">
  </fieldset>

  <!-- Submission -->
  <button type="submit">Submit</button>
  <button type="reset">Clear</button>
</form>

<!-- File uploads need enctype -->
<form method="post" enctype="multipart/form-data" action="upload.php">
  <input type="file" name="photo">
  <input type="submit" value="Upload">
</form>
```

Tip: GET appends data to the URL (bookmarkable). POST sends data in the request body (more secure). Use POST for sensitive data and file uploads.

CHAPTER 14

Global Attributes

Definition: Global attributes can be applied to ANY HTML element. They include id, class, style, title, lang, tabindex, contenteditable, draggable, hidden, and more.

Try It:

```
<!-- contenteditable: user can edit the text inline -->
<p contenteditable="true">Click me to edit this text!</p>

<!-- hidden: removes from layout and accessibility tree -->
<div hidden>This is not visible on the page.</div>

<!-- draggable: enables drag-and-drop -->
<img src="logo.png" draggable="true" alt="Drag me">

<!-- title: tooltip on hover -->
<abbr title="HyperText Markup Language">HTML</abbr>

<!-- dir: text direction -->
<p dir="rtl">This text goes right-to-left (Arabic/Hebrew).</p>

<!-- spellcheck -->
<textarea spellcheck="true">Check my spelling here...</textarea>

<!-- translate: hint for translation tools -->
<span translate="no">Brand Name XYZ</span>
```

Tip: The style and class attributes are the most commonly used global attributes for applying CSS to individual elements.

CHAPTER 15

Headings

Definition: HTML provides 6 heading levels: <h1> (most important) through <h6> (least important). Headings define the document outline and are critical for accessibility and SEO. Use them in hierarchical order.

Try It:

```
<!-- All six heading levels -->
<h1>Page Title (only one per page)</h1>
<h2>Main Section</h2>
<h3>Subsection</h3>
<h4>Sub-subsection</h4>
<h5>Minor heading</h5>
<h6>Smallest heading</h6>

<!-- Correct document structure -->
<h1>My Blog</h1>
  <h2>Technology Posts</h2>
    <h3>Why I Love HTML</h3>
    <p>Paragraph content...</p>
    <h3>CSS Tips</h3>
    <p>More content...</p>
  <h2>Travel Posts</h2>
    <h3>My Trip to Paris</h3>
```

Tip: Never skip heading levels (e.g., jumping from h1 to h4). Screen readers use headings for navigation. Only one <h1> per page.

CHAPTER 16

HTML5 Cache (AppCache)

Definition: The HTML5 Application Cache (manifest) lets browsers store specified files offline. A .appcache manifest file lists which resources to cache, which require a network, and what to show as fallback. Note: AppCache is deprecated — use Service Workers in modern apps.

Try It:

```
<!-- index.html: reference the manifest -->
<!DOCTYPE html>
<html manifest="app.appcache">
<body>
  <h1>Offline-Capable Page</h1>
</body>
</html>

<!-- app.appcache manifest file -->
CACHE MANIFEST
# Version 1.0

CACHE:
index.html
style.css
app.js
logo.png

NETWORK:
api/data    # always fetched from network

FALLBACK:
/ offline.html   # shown when network unavailable
```

Tip: AppCache is deprecated. For production offline apps, use Service Workers with the Cache API (modern standard).

CHAPTER 17

HTML Event Attributes

Definition: Event attributes attach inline JavaScript handlers to HTML elements that execute when specific events occur (click, input, submit, keydown, etc.).

Try It:

```
<!-- Form events -->
<input type="text"
       onfocus="this.style.background='lightyellow'"
       onblur="this.style.background=''"
       onchange="console.log('changed:', this.value)"
       oninput="document.getElementById('out').textContent=this.value">
<p id="out"></p>

<!-- Keyboard events -->
<input type="text"
       onkeydown="console.log('down:', event.key)"
       onkeyup="console.log('up:', event.key)">

<!-- Form submission event -->
<form onsubmit="event.preventDefault(); alert('Submitted!');">
  <input type="text" name="q">
  <button type="submit">Go</button>
</form>

<!-- Mouse events -->
<button onclick="alert('Clicked!')"
        onmouseover="this.style.color='blue'"
        onmouseout="this.style.color=''">
  Hover and Click Me
</button>
```

Tip: Prefer addEventListener() in JavaScript over inline event attributes — it keeps HTML and JS separate and allows multiple handlers.

CHAPTER 18

IFrames

Definition: An <iframe> (inline frame) embeds another HTML document inside the current page. Common uses include embedding maps, videos, and third-party widgets. Subject to the Same-Origin Policy for JavaScript access.

Try It:

```
<!-- Basic iframe -->
<iframe src="https://example.com" width="800" height="400"></iframe>

<!-- Embed a YouTube video -->
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/VIDEO_ID"
  allowfullscreen>
</iframe>

<!-- Inline content with srcdoc -->
<iframe srcdoc="<h1>Hello from srcdoc!</h1><p>Embedded HTML.</p>"
        width="400" height="200">
</iframe>

<!-- Sandboxed iframe (restrictions) -->
<iframe src="untrusted.html"
        sandbox="allow-scripts allow-same-origin">
</iframe>

<!-- Navigate iframe from outside using anchor target -->
<iframe name="myFrame" src="page1.html" width="600" height="400"></iframe>
<a href="page2.html" target="myFrame">Load Page 2</a>
```

Tip: The sandbox attribute disables scripts, forms, and popups by default. Re-enable specific permissions using space-separated values like allow-scripts.

CHAPTER 19

Image Maps

Definition: An image map defines clickable areas (hotspots) on an image. Each area can act as a separate hyperlink. Bind the image to the map with the usemap attribute matching the map's name.

Try It:

```
<!-- Image with a map -->
<img src="world.png" alt="World Map" usemap="#worldmap" width="800"
height="400">

<map name="worldmap">
  <!-- Rectangle: left, top, right, bottom -->
  <area shape="rect"
        coords="50,80,200,200"
        href="europe.html"
        alt="Europe">

  <!-- Circle: centerX, centerY, radius -->
  <area shape="circle"
        coords="400,180,60"
        href="africa.html"
        alt="Africa">

  <!-- Polygon: x1,y1, x2,y2, x3,y3 ... -->
  <area shape="poly"
        coords="600,100,700,50,750,150,650,180"
        href="asia.html"
        alt="Asia">

  <!-- Default: catches clicks outside other areas -->
  <area shape="default" href="other.html" alt="Other">
</map>
```

Tip: Use an image map editor tool (like image-map.net) to easily calculate pixel coordinates for complex shapes.

CHAPTER 20

Images

Definition: The <img> element embeds images. It is a void element (self-closing). The src attribute provides the image source, and alt provides accessible alternative text for screen readers and when images fail to load.

Try It:

```
<!-- Basic image -->
<img src="photo.jpg" alt="A scenic mountain landscape" width="800" height="600">

<!-- Image as link -->
<a href="gallery.html">
  <img src="thumbnail.jpg" alt="Click to view gallery">
</a>

<!-- Responsive image with srcset (different resolutions) -->
<img src="photo-300.jpg"
     srcset="photo-300.jpg 1x, photo-600.jpg 2x, photo-900.jpg 3x"
     alt="Responsive photo">

<!-- srcset with sizes (art direction) -->
<img src="default.jpg"
     srcset="small.jpg 480w, medium.jpg 800w, large.jpg 1200w"
     sizes="(max-width:600px) 480px, (max-width:900px) 800px, 1200px"
     alt="Art directed image">

<!-- Picture element for responsive images -->
<picture>
  <source media="(min-width:800px)" srcset="large.jpg">
  <source media="(min-width:400px)" srcset="medium.jpg">
  <img src="small.jpg" alt="Responsive picture element">
</picture>

<!-- Inline base64 image (small icons) -->
<img src="..." alt="Icon">
```

Tip: Always include a meaningful alt attribute. Empty alt="" is correct for decorative images. Width/height attributes prevent layout shifts.

CHAPTER 21

Include JavaScript Code in HTML

Definition: JavaScript can be included in HTML three ways: inline in a <script> tag, in an external .js file via the src attribute, or asynchronously/deferred to control load order.

Try It:

```
<!-- 1. External script (best practice — put before </body>) -->
<script src="app.js"></script>

<!-- 2. Inline script -->
<script>
  console.log("Page loaded!");
  document.getElementById("demo").textContent = "Changed by JS";
</script>

<!-- 3. Async: downloads in parallel, executes immediately when done -->
<script src="analytics.js" async></script>

<!-- 4. Defer: downloads in parallel, executes after HTML is parsed -->
<script src="app.js" defer></script>

<!-- 5. Module script (ES6) -->
<script type="module" src="main.mjs"></script>

<!-- 6. Fallback when JS is disabled -->
<script>
  document.write("JavaScript is active!");
</script>
<noscript>
  <p>JavaScript is disabled. Please enable it for full functionality.</p>
</noscript>
```

Tip: Use defer for most scripts. Use async for independent scripts (analytics, ads). Modules are deferred by default.

CHAPTER 22

Input Control Elements

Definition: The <input> element creates interactive form controls. Its type attribute determines behavior: text, email, password, checkbox, radio, file, number, range, date, color, and more.

Try It:

```
<form>
  <!-- Text inputs -->
  <input type="text"     placeholder="Name"     name="name">
  <input type="email"    placeholder="Email"    name="email">
  <input type="password" placeholder="Password" name="pwd">
  <input type="tel"      placeholder="Phone"    name="tel">
  <input type="url"      placeholder="Website"  name="url">
  <input type="search"   placeholder="Search"   name="q">

  <!-- Number controls -->
  <input type="number" min="1" max="100" step="1" value="10">
  <input type="range"  min="0" max="100" step="5" value="50">

  <!-- Date / Time -->
  <input type="date">
  <input type="time">
  <input type="datetime-local">
  <input type="month">
  <input type="week">

  <!-- Color picker -->
  <input type="color" value="#1565C0">

  <!-- Checkboxes -->
  <input type="checkbox" id="agree" name="agree" checked>
  <label for="agree">I agree</label>

  <!-- Radio buttons (same name = one group) -->
  <input type="radio" name="size" value="s" id="sm">
  <label for="sm">Small</label>
  <input type="radio" name="size" value="m" id="md" checked>
  <label for="md">Medium</label>

  <!-- File upload -->
  <input type="file" accept="image/*" multiple>

  <!-- Hidden field -->
  <input type="hidden" name="token" value="abc123">
```

```
  <!-- Validation attributes -->
  <input type="text" required minlength="3" maxlength="50">
  <input type="text" pattern="[A-Z]{3}" title="3 uppercase letters">

  <!-- Buttons -->
  <input type="submit" value="Submit">
  <input type="reset"  value="Clear">
  <input type="button" value="Click Me" onclick="alert('Hi!')">
</form>
```

 Tip: HTML5 input types provide built-in validation and specialized keyboards on mobile (e.g., type="email" shows @ on mobile keyboard).

CHAPTER 23

Label Element

Definition: The <label> element associates a text description with a form control. Clicking the label focuses or activates its associated input. This is critical for accessibility.

Try It:

```
<!-- Method 1: for attribute links to input id -->
<label for="username">Username:</label>
<input type="text" id="username" name="username">

<!-- Method 2: wrapping (implicit association) -->
<label>
  Password:
  <input type="password" name="password">
</label>

<!-- Labels with checkboxes (click label to toggle checkbox) -->
<label>
  <input type="checkbox" name="newsletter">
  Subscribe to newsletter
</label>

<!-- Labels with radio buttons -->
<fieldset>
  <legend>Preferred contact method:</legend>
  <label><input type="radio" name="contact" value="email"> Email</label>
  <label><input type="radio" name="contact" value="phone"> Phone</label>
</fieldset>

<!-- Label outside the form (using form attribute) -->
<form id="myForm" action="/go">
  <input id="qty" type="number" name="qty">
  <button type="submit">Order</button>
</form>
<label for="qty" form="myForm">Quantity:</label>
```

Tip: Every form input should have a <label>. This improves accessibility (screen readers) and usability (larger click target).

CHAPTER 24

Linking Resources

Definition: The <link> element in the <head> connects external resources (stylesheets, icons, fonts, feeds) to the document. The rel attribute specifies the relationship.

Try It:

```
<head>
  <!-- CSS Stylesheet -->
  <link rel="stylesheet" href="style.css">

  <!-- Favicon -->
  <link rel="icon" type="image/png" href="/favicon.png">
  <link rel="shortcut icon" type="image/x-icon" href="/favicon.ico">

  <!-- Google Fonts -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="stylesheet"
        href="https://fonts.googleapis.com/css2?family=Roboto&display=swap">

  <!-- Print-specific stylesheet -->
  <link rel="stylesheet" href="print.css" media="print">

  <!-- RSS / Atom feed -->
  <link rel="alternate" type="application/rss+xml" href="/feed.xml">

  <!-- Performance hints -->
  <link rel="dns-prefetch"  href="https://cdn.example.com">
  <link rel="preconnect"    href="https://cdn.example.com">
  <link rel="prefetch"      href="next-page.html">
  <link rel="prerender"     href="landing.html">

  <!-- Canonical URL -->
  <link rel="canonical" href="https://example.com/page/">

  <!-- Prev / Next in a series -->
  <link rel="prev" href="chapter1.html">
  <link rel="next" href="chapter3.html">
</head>
```

Tip: Place all <link> tags inside <head>. dns-prefetch and preconnect speed up external resource loading noticeably.

CHAPTER 25

Lists

Definition: HTML has three list types: <ul> (unordered / bulleted), <ol> (ordered / numbered), and <dl> (description list of term-definition pairs). Lists can be nested inside each other.

Try It:

```
<!-- Unordered list -->
<ul>
  <li>Apples</li>
  <li>Bananas</li>
  <li>Cherries</li>
</ul>

<!-- Ordered list (default: 1, 2, 3) -->
<ol>
  <li>Preheat oven</li>
  <li>Mix ingredients</li>
  <li>Bake 30 minutes</li>
</ol>

<!-- Ordered list options -->
<ol type="A" start="3">     <!-- C, D, E... -->
  <li>Third item as C</li>
</ol>
<ol reversed>               <!-- Counts down -->
  <li>Item 3</li>
  <li>Item 2</li>
  <li>Item 1</li>
</ol>

<!-- Description list -->
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language — structures web content.</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets — styles web content.</dd>
</dl>

<!-- Nested lists -->
<ul>
  <li>Fruits
    <ul>
      <li>Citrus
        <ul><li>Orange</li><li>Lemon</li></ul>
      </li>
    </ul>
```

```
    </li>
  </ul>
```

Tip: type="A" → A,B,C type="a" → a,b,c type="I" → I,II,III type="i" → i,ii,iii

CHAPTER 26

Marking Up Computer Code

Definition: Use <code> for inline code snippets and <pre><code> for multi-line code blocks. Related elements: <var> for variables, <samp> for computer output, <kbd> for keyboard input.

Try It:

```
<!-- Inline code -->
<p>Use the <code>console.log()</code> function to debug JavaScript.</p>
<p>The <code>&lt;img&gt;</code> tag is a void element.</p>

<!-- Multi-line code block -->
<pre>
  <code>
function greet(name) {
  return "Hello, " + name + "!";
}
console.log(greet("World"));
  </code>
</pre>

<!-- Variable -->
<p>The formula is: <var>E</var> = <var>m</var><var>c</var><sup>2</sup></p>

<!-- Computer output -->
<p>Running the script produced: <samp>Error: file not found</samp></p>

<!-- Keyboard input -->
<p>Press <kbd>Ctrl</kbd> + <kbd>C</kbd> to copy.</p>

<!-- Must escape < and > inside code blocks -->
<pre><code>&lt;p&gt;This is HTML code displayed as text&lt;/p&gt;</code></pre>
```

Tip: Always escape < as < and > as > inside <code> blocks. Many developers add a JavaScript syntax highlighter like Prism.js for colored code.

CHAPTER 27

Marking Up Quotes

Definition: <q> is for short inline quotes; <blockquote> is for longer block-level quotes. The cite attribute on both specifies the source URL. The <cite> element marks the title of a creative work.

Try It:

```
<!-- Inline quote (browser adds quotation marks automatically) -->
<p>Einstein wrote <q>Imagination is more important than knowledge.</q></p>

<!-- Inline quote with source -->
<p>She argued <q cite="https://example.com/paper">data drives decisions</q></p>

<!-- Block quote -->
<blockquote cite="https://example.com/article">
  <p>The greatest glory in living lies not in never falling,
     but in rising every time we fall.</p>
</blockquote>
<p>— <cite>Nelson Mandela</cite></p>

<!-- Block quote with footer attribution (HTML5) -->
<blockquote cite="https://www.w3.org/">
  <p>The Web is for everyone.</p>
  <footer>
    — <cite><a href="https://www.w3.org/" rel="external">W3C</a></cite>
  </footer>
</blockquote>

<!-- Cite element: title of a work -->
<p>I just finished reading <cite>The Great Gatsby</cite>.</p>
```

Tip: Do NOT add quotation marks manually around <q> — browsers insert them automatically. <cite> is for titles, not author names.

CHAPTER 28

Media Elements

Definition: HTML5 introduced native <audio> and <video> elements for embedding media without plugins. Multiple <source> children provide fallback formats for browser compatibility.

Try It:

```
<!-- Audio with controls and fallback sources -->
<audio controls>
  <source src="song.ogg" type="audio/ogg">
  <source src="song.mp3" type="audio/mpeg">
  Your browser does not support audio.
</audio>

<!-- Audio attributes -->
<audio src="music.mp3" controls autoplay loop muted></audio>

<!-- Video with controls -->
<video width="640" height="360" controls poster="thumbnail.jpg">
  <source src="video.webm" type="video/webm">
  <source src="video.mp4"  type="video/mp4">
  <source src="video.ogv"  type="video/ogg">
  Your browser does not support video.
</video>

<!-- Video with subtitles -->
<video controls>
  <source src="movie.mp4" type="video/mp4">
  <track kind="subtitles" src="subs-en.vtt" srclang="en" label="English"
default>
  <track kind="subtitles" src="subs-fr.vtt" srclang="fr" label="Français">
</video>

<!-- Background/header video (no controls, auto-plays muted) -->
<video autoplay muted loop id="bgvideo" poster="bg.jpg">
  <source src="bg.mp4" type="video/mp4">
</video>
```

Tip: Always provide multiple formats (mp4 + webm) for maximum browser compatibility. Use muted for autoplay — browsers block unmuted autoplay.

CHAPTER 29

Meta Information

Definition: <meta> tags in the <head> provide metadata about the document — character set, viewport, description, keywords, author, and social media sharing information. They do not render visually.

Try It:

```
<head>
  <!-- Character encoding (always first) -->
  <meta charset="UTF-8">

  <!-- Responsive design viewport -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- SEO basics -->
  <meta name="description" content="Learn HTML from basic to advanced
concepts.">
  <meta name="keywords" content="HTML, CSS, web development, tutorial">
  <meta name="author" content="Jane Developer">

  <!-- Robots directive -->
  <meta name="robots" content="index, follow">

  <!-- Auto-refresh (use sparingly) -->
  <meta http-equiv="refresh" content="30">

  <!-- Auto-redirect after 5 seconds -->
  <meta http-equiv="refresh" content="5;url=https://newsite.com/">

  <!-- Disable phone number detection on iOS -->
  <meta name="format-detection" content="telephone=no">

  <!-- Open Graph (Facebook / LinkedIn sharing) -->
  <meta property="og:title" content="My Page Title">
  <meta property="og:description" content="A brief description.">
  <meta property="og:image" content="https://example.com/image.jpg">
  <meta property="og:url" content="https://example.com/page">
  <meta property="og:type" content="website">

  <!-- Twitter Cards -->
  <meta name="twitter:card" content="summary_large_image">
  <meta name="twitter:title" content="My Page Title">
  <meta name="twitter:description" content="A brief description.">
  <meta name="twitter:image" content="https://example.com/image.jpg">

  <!-- Web App (PWA) -->
```

```
  <meta name="mobile-web-app-capable" content="yes">
  <meta name="theme-color" content="#1565C0">
</head>
```

 Tip: The viewport meta tag is required for responsive design. Without it, mobile browsers will zoom out the desktop layout.

CHAPTER 30

Navigation Bars

Definition: Navigation bars are collections of links that let users move between sections or pages. HTML5 provides the semantic <nav> element. Style with CSS to create horizontal, vertical, or dropdown menus.

Try It:

```
<!-- Semantic HTML5 navigation -->
<nav role="navigation" aria-label="Main navigation">
  <ul>
    <li><a href="/" aria-current="page">Home</a></li>
    <li><a href="/about">About</a></li>
    <li><a href="/services">Services</a></li>
    <li><a href="/contact">Contact</a></li>
  </ul>
</nav>

<!-- CSS for horizontal navbar -->
<style>
nav ul { list-style: none; display: flex; gap: 16px; padding: 0; margin: 0;
         background: #1565C0; }
nav ul li a { color: white; text-decoration: none; padding: 12px 20px;
              display: block; }
nav ul li a:hover, nav ul li a[aria-current="page"] {
  background: #0D47A1; }
</style>

<!-- Simple inline navigation (no list) -->
<nav>
  <a href="/">Home</a>
  <a href="/blog">Blog</a>
  <a href="/contact">Contact</a>
</nav>
```

Tip: Use aria-current="page" on the active link to indicate the current page for screen readers. Use <nav> only for major navigation blocks.

CHAPTER 31

Output Element

Definition: The <output> element represents the result of a calculation or user action. It is associated with form inputs via the for attribute and belongs to a form via the form attribute.

Try It:

```
<!-- Live calculator using output -->
<form oninput="result.value = parseInt(a.value) + parseInt(b.value)">
  <label>Number A: <input type="number" id="a" name="a" value="0"></label>
  +
  <label>Number B: <input type="number" id="b" name="b" value="0"></label>
  =
  <output name="result" for="a b">0</output>
</form>

<!-- Output with range slider -->
<form oninput="vol.value = volume.value">
  <label>
    Volume: <input type="range" id="volume" name="volume"
                   min="0" max="100" value="50">
  </label>
  <output name="vol" for="volume">50</output>
</form>

<!-- Output outside its form -->
<form id="calc" oninput="res.value = x.value * y.value">
  <input type="number" id="x" value="3"> ×
  <input type="number" id="y" value="4">
</form>
<output name="res" for="x y" form="calc">12</output>
```

Tip: The <output> element updates live via form events. Use oninput on the <form> for real-time recalculation as users type.

CHAPTER 32

Paragraphs

Definition: <p> defines a paragraph of text. The browser adds spacing before and after.
 inserts a line break without starting a new paragraph. <pre> preserves whitespace and line breaks exactly as written.

Try It:

```
<!-- Basic paragraphs -->
<p>This is the first paragraph. It flows naturally.</p>
<p>This is the second paragraph. Browsers add space between them.</p>

<!-- Line break (no new paragraph) -->
<p>
  Line one.<br>
  Line two on same paragraph.<br>
  Line three.
</p>

<!-- Extra spaces in HTML are ignored — only one space renders -->
<p>This     has    lots    of    spaces.</p>
<!-- Renders as: "This has lots of spaces." -->

<!-- Pre-formatted text (preserves whitespace) -->
<pre>
  Name:   Alice
  Score:  98
  Grade:  A
</pre>

<!-- Combining p and br (poems, addresses) -->
<p>
  123 Main Street,<br>
  Springfield, IL 62701<br>
  United States
</p>
```

Tip: Never use
 tags just to add vertical space between elements — use CSS margin/padding instead.
 is for intentional line breaks in content.

CHAPTER 33

Progress Element

Definition: The <progress> element displays a progress bar showing task completion. The value attribute is the current progress; max is the total. Without value, it shows an indeterminate/animated state.

Try It:

```
<!-- Determinate progress (22 out of 100) -->
<progress value="22" max="100">22%</progress>

<!-- Indeterminate (loading, unknown duration) -->
<progress max="100"></progress>

<!-- JavaScript-driven progress bar -->
<progress id="bar" value="0" max="100">0%</progress>
<button onclick="startProgress()">Start Task</button>

<script>
function startProgress() {
  let bar = document.getElementById("bar");
  let val = 0;
  let timer = setInterval(() => {
    val += 5;
    bar.value = val;
    if (val >= 100) clearInterval(timer);
  }, 200);
}
</script>

<!-- Styling progress (cross-browser) -->
<style>
progress { width: 300px; height: 20px; }
/* Chrome/Safari */
progress::-webkit-progress-bar  { background: #eee; border-radius: 4px; }
progress::-webkit-progress-value { background: #1565C0; border-radius: 4px; }
/* Firefox */
progress::-moz-progress-bar { background: #1565C0; }
</style>
```

Tip: Use <meter> (not <progress>) to display a static gauge (like disk usage). <progress> is specifically for task completion state.

CHAPTER 34

Sectioning Elements

Definition: HTML5 sectioning elements define the document structure semantically: <header>, <nav>, <main>, <section>, <article>, <aside>, and <footer>. They replace generic wrappers with meaningful markup.

Try It:

```
<body>
  <header>
    <h1>My Blog</h1>
    <nav>
      <a href="/">Home</a>
      <a href="/about">About</a>
    </nav>
  </header>

  <main>
    <!-- Self-contained, distributable content -->
    <article>
      <header>
        <h2>Blog Post Title</h2>
        <time datetime="2024-01-15">January 15, 2024</time>
      </header>
      <p>Introduction paragraph.</p>

      <!-- Thematic grouping within the article -->
      <section>
        <h3>Chapter 1</h3>
        <p>Content of chapter 1...</p>
      </section>
      <section>
        <h3>Chapter 2</h3>
        <p>Content of chapter 2...</p>
      </section>
    </article>

    <!-- Tangentially related content -->
    <aside>
      <h3>Related Posts</h3>
      <ul>...<ul>
    </aside>
  </main>

  <footer>
    <p>&copy; 2024 My Blog. All rights reserved.</p>
  </footer>
</body>
```

 Tip: <main> must appear only once per page. <article> is for independently-distributable content. <section> requires a heading. <aside> is for tangentially related content.

CHAPTER 35

Selection Menu Controls

Definition: <select> creates a dropdown menu; <option> defines each choice; <optgroup> groups related options. The <datalist> element provides autocomplete suggestions for an <input> field.

Try It:

```
<!-- Basic dropdown -->
<select name="country" id="country">
  <option value="">-- Select a country --</option>
  <option value="us">United States</option>
  <option value="uk">United Kingdom</option>
  <option value="in" selected>India</option>
</select>

<!-- Option groups -->
<select name="cuisine">
  <optgroup label="Asian">
    <option value="indian">Indian</option>
    <option value="japanese">Japanese</option>
  </optgroup>
  <optgroup label="European">
    <option value="italian">Italian</option>
    <option value="french">French</option>
  </optgroup>
</select>

<!-- Multi-select (Ctrl/Cmd + click to select multiple) -->
<select name="skills" multiple size="4">
  <option value="html" selected>HTML</option>
  <option value="css">CSS</option>
  <option value="js" selected>JavaScript</option>
  <option value="python">Python</option>
</select>

<!-- Datalist: autocomplete suggestions on an input -->
<input list="browsers" name="browser" placeholder="Choose browser">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari">
  <option value="Edge">
</datalist>
```

Tip: datalist is not supported in Safari (iOS). Use a polyfill or a custom dropdown component for full cross-browser support of autocomplete.

CHAPTER 36

SVG (Scalable Vector Graphics)

Definition: SVG is an XML-based format for scalable, resolution-independent vector graphics. Unlike raster images (PNG/JPEG), SVG images remain crisp at any size. SVG can be inline HTML, external files, or CSS backgrounds.

Try It:

```
<!-- Inline SVG (can be styled and scripted with CSS/JS) -->
<svg xmlns="http://www.w3.org/2000/svg" width="200" height="200"
     viewBox="0 0 200 200">
  <!-- Circle -->
  <circle cx="100" cy="100" r="80" fill="#1565C0" stroke="#0D47A1" stroke-
width="4"/>
  <!-- Rectangle -->
  <rect x="60" y="60" width="80" height="80" fill="white" opacity="0.5"/>
  <!-- Text -->
  <text x="100" y="108" text-anchor="middle" fill="white"
        font-size="18" font-family="Arial">SVG!</text>
</svg>

<!-- External SVG as image (cannot be styled by CSS) -->
<img src="icon.svg" alt="Icon" width="50" height="50">

<!-- External SVG as object (can be scripted) -->
<object type="image/svg+xml" data="chart.svg" width="400" height="300">
  Fallback text if SVG not supported.
</object>

<!-- SVG as CSS background -->
<style>
.icon {
  width: 50px; height: 50px;
  background: url(icon.svg) no-repeat center/contain;
}
</style>
<div class="icon"></div>
```

Tip: Inline SVG gives you full CSS and JavaScript control. Use <img src="...svg"> for simple display. Use SVG for icons, logos, and charts on HiDPI/Retina screens.

CHAPTER 37

Tabindex

Definition: The tabindex attribute controls keyboard Tab navigation order. tabindex="0" includes an element in the natural tab flow. tabindex="-1" removes it from tab order (but keeps it focusable via JavaScript). Positive values set an explicit order.

Try It:

```
<!-- Add non-interactive element to tab order -->
<div tabindex="0" onclick="activate(this)">
  Click or Tab to me
</div>

<!-- Remove from tab order (focusable by JS only) -->
<button tabindex="-1" id="modalClose">Close Modal</button>
<!-- Can be focused programmatically: -->
<script>document.getElementById("modalClose").focus();</script>

<!-- Custom tab order (positive values — avoid this!) -->
<input tabindex="3" placeholder="Third">
<input tabindex="1" placeholder="First">
<input tabindex="2" placeholder="Second">

<!-- Better: use natural DOM order instead -->
<input placeholder="First">
<input placeholder="Second">
<input placeholder="Third">

<!-- Focusable custom component -->
<div role="button" tabindex="0"
     onclick="doAction()"
     onkeydown="if(event.key==='Enter'||event.key===' ') doAction()">
  Custom Button
</div>
```

Tip: Avoid positive tabindex values — they break the natural tab flow and confuse users. Rely on DOM order for focus sequence.

CHAPTER 38

Tables

Definition: HTML tables display two-dimensional tabular data in rows and columns. Key elements: <table>, <thead>, <tbody>, <tfoot>, <tr>, <th> (header cell), <td> (data cell), and <caption>. Tables are NOT for layout.

▶ **Try It:**

```
<table>
  <caption>Student Scores</caption>
  <thead>
    <tr>
      <th scope="col">Name</th>
      <th scope="col">Math</th>
      <th scope="col">Science</th>
      <th scope="col">Average</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Alice</th>
      <td>95</td>
      <td>88</td>
      <td>91.5</td>
    </tr>
    <tr>
      <th scope="row">Bob</th>
      <td>78</td>
      <td>85</td>
      <td>81.5</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th>Class Average</th>
      <td>86.5</td>
      <td>86.5</td>
      <td>86.5</td>
    </tr>
  </tfoot>
</table>

<!-- Spanning cells -->
<table>
  <tr>
    <td colspan="2">Spans 2 columns</td>
    <td>Col 3</td>
  </tr>
  <tr>
```

```
    <td rowspan="2">Spans 2 rows</td>
    <td>Row 2, Col 2</td>
    <td>Row 2, Col 3</td>
  </tr>
  <tr>
    <td>Row 3, Col 2</td>
    <td>Row 3, Col 3</td>
  </tr>
</table>
```

Tip: Use scope="col" on column headers and scope="row" on row headers for screen reader accessibility. Add CSS border-collapse: collapse for clean table borders.

CHAPTER 39

Text Formatting

Definition: HTML provides semantic text formatting tags: (bold, important), (italic, emphasis), <mark> (highlight), <del> (deleted), <ins> (inserted), <sup>/<sub> (super/subscript), and more.

Try It:

```
<!-- Bold -->
<strong>Semantically important text</strong>  <!-- bold + important -->
<b>Stylistically bold only</b>               <!-- visual only -->

<!-- Italic -->
<em>Emphasized text (stress)</em>            <!-- italic + stressed -->
<i>Stylistically italic (term, title)</i>   <!-- visual only -->

<!-- Underline (HTML5: annotation) -->
<u>Misspelled or annotated text</u>

<!-- Highlight (HTML5) -->
<p>Search results contain <mark>HTML</mark> tutorials.</p>

<!-- Strikethrough / Delete / Insert -->
<p>Price: <del>$50</del> <ins>$35</ins></p>
<s>This fact is no longer accurate.</s>

<!-- Superscript and Subscript -->
<p>E = mc<sup>2</sup></p>
<p>H<sub>2</sub>O is water</p>
<p>CO<sub>2</sub> = Carbon Dioxide</p>

<!-- Abbreviation with tooltip -->
<p><abbr title="HyperText Markup Language">HTML</abbr> is fun!</p>

<!-- Small print -->
<p><small>* Terms and conditions apply.</small></p>
```

Tip: Use and for semantic meaning (screen readers announce them differently). Use <b> and <i> only for stylistic text with no semantic importance.

CHAPTER 40

Using HTML with CSS

Definition: CSS (Cascading Style Sheets) styles HTML elements. There are three ways to apply CSS: external stylesheet (recommended), internal <style> block, and inline style attribute. Later declarations override earlier ones.

Try It:

```
<!-- 1. EXTERNAL STYLESHEET (best practice) -->
<head>
  <link rel="stylesheet" href="styles.css">
</head>

<!-- styles.css -->
body { font-family: Arial, sans-serif; margin: 0; padding: 0; }
h1   { color: #1565C0; }
.card { border: 1px solid #ccc; padding: 16px; border-radius: 8px; }

<!-- 2. INTERNAL STYLESHEET -->
<head>
  <style>
    body { background-color: #f5f5f5; }
    p    { line-height: 1.6; color: #333; }
  </style>
</head>

<!-- 3. INLINE STYLE (avoid unless necessary) -->
<p style="color: red; font-size: 18px;">Inline styled paragraph</p>

<!-- Multiple external stylesheets (last wins) -->
<link rel="stylesheet" href="base.css">
<link rel="stylesheet" href="theme.css">
<link rel="stylesheet" href="overrides.css">

<!-- CDN stylesheet (Bootstrap example) -->
<link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
```

Tip: Cascade order (highest to lowest priority): inline style > internal <style> > external stylesheet > browser defaults. Specificity and !important also affect this order.

CHAPTER 41

Void Elements

Definition: Void elements are HTML elements that cannot have any child content and therefore do not have a closing tag. They may have attributes but are self-contained. HTML5 does not require the self-closing slash (/>), but it is allowed.

Complete List of HTML5 Void Elements

<area> <base>
 <col> <embed> <hr> <img> <input> <link> <meta> <param> <source> <track> <wbr>

Try It:

```
<!-- <br>: Line break -->
<p>Line one.<br>Line two.</p>

<!-- <hr>: Horizontal rule (thematic break) -->
<section>Introduction</section>
<hr>
<section>Main content</section>

<!-- <img>: Image (self-closing) -->
<img src="photo.jpg" alt="A beautiful landscape" width="800" height="600">

<!-- <input>: Form control -->
<input type="text" name="username" placeholder="Enter username">

<!-- <link>: External resource -->
<link rel="stylesheet" href="style.css">

<!-- <meta>: Metadata -->
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">

<!-- <source>: Media source -->
<video controls>
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
</video>

<!-- <track>: Text track for media -->
<video>
  <track kind="subtitles" src="subs.vtt" srclang="en" label="English">
</video>
```

```
<!-- <wbr>: Word break opportunity (hint where to break a long word) -->
<p>superlongURL/<wbr>very/<wbr>deep/<wbr>path/<wbr>file.html</p>

<!-- <base>: Default URL for relative links -->
<head>
  <base href="https://example.com/" target="_blank">
</head>
```

 Tip: In HTML5,
 and
 are both valid. Self-closing syntax is required in XHTML/XML. Consistency is key — pick one style and stick to it.

End of Student Guide · HTML Chapters 1 – 41